R2.07 : TD 4

Ordonnancement

23 février 2026 – B. Colombel

Nous connaissons déjà deux algorithmes pour rechercher une plus courte chaîne dans un graphe pondéré :
- l'algorithme de Dijkstra pour les graphes pondérés à valuations positives ;
- l'algorithme de Bellman-Ford pour les graphes pondérés dont les valuations peuvent être négatives et sans circuit « négatifs ».

Ces deux algorithmes peuvent être améliorés dans le cas de graphes pondérés sans circuit.

1 Graphe sans circuit

1.1 Décomposition en niveau

Les graphes orientés **sans circuit** possèdent des propriétés spécifiques : en particulier il existe dans un graphe sans circuit une notion de hiérarchie entre les sommets. C'est ce qu'on appelle la **décomposition en niveaux** ou aussi un « tri topologique »

Théorème 1 (Décomposition en niveaux d'un graphe sans circuit)
Si G est un graphe sans circuit, alors on peut définir pour chaque sommet un niveau de la manière suivante :
- les sommets sans prédécesseurs sont de rang 0 ;
- tout sommet x a un rang supérieur aux rangs de ses prédécesseurs :

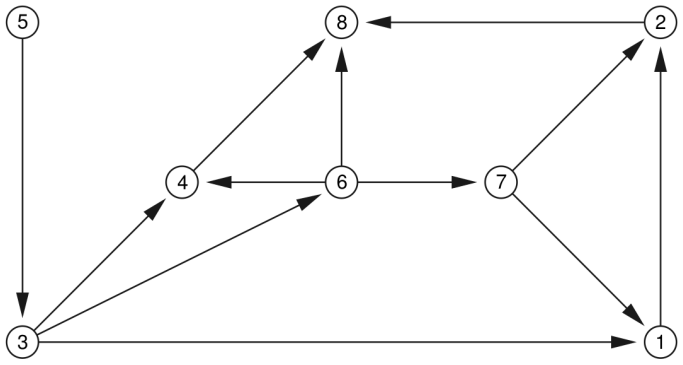
$$\text{rang}(x) = \max_{y \in N^-(x)} \text{rang}(y) + 1.$$

où $N^-(x)$ est l'ensemble des prédécesseurs du sommet x.

Remarque. Cela signifie que :
- les sommets de rang 0 sont les sommets qui n'ont pas de prédécesseurs ;
- les sommets de rang 1 sont les sommets qui n'ont que des sommets de rang 0 comme prédécesseurs ;
- les sommets de rang 2 sont les sommets qui n'ont que des sommets de rang inférieur ou égaux à 1 comme prédécesseurs ;
- etc.

Pour déterminer les niveaux des sommets d'un graphe de manière systématique, c'est-à-dire algorithmique, on peut utiliser la **matrice des prédécesseurs** (ou matrice d'**antériorité**) comme le montre l'exercice suivant :

Exercice 1. Considérons le graphe orienté suivant :



1. La matrice ci-dessous est la première matrice des prédécesseurs :

$$\begin{pmatrix} 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 & 0 \end{pmatrix} \qquad \text{définie par } a_{i,j} = \begin{cases} 1 \text{ si } j \text{ est un prédécesseur de } i \\ 0 \text{ sinon} \end{cases}$$

   (a) Effectuer la somme des coefficients de la matrice sur chacune de ses lignes. Que signifient les résultats obtenus ?

   En déduire les sommets de rang 0.

   (b) Annuler tous les coefficients des colonnes des sommets de rang 0 et déterminer de même la somme des coefficients sur chacune des lignes. Que signifient les résultats obtenus ?

   En déduire les sommets de rang 1.

   (c) Recommencer le processus jusqu'à obtenir les rangs de tous les sommets du graphes. Vous pourrez vous aider des matrices vides ci-dessous.

$$\begin{pmatrix} \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \end{pmatrix} \qquad \begin{pmatrix} \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \end{pmatrix}$$

$$\begin{pmatrix} \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \end{pmatrix} \qquad \begin{pmatrix} \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \end{pmatrix}$$

2. Redessiner le graphe en utilisant la numérotation topologique.

Remarques. Si G possède un circuit $C = (x_0,x_1,...,x_{n-1},x_0)$, il ne peut admettre de décomposition en niveaux, puisque l'on aurait alors la propriété :

$$\text{rang}(x_0) < \text{rang}(x_1) < ... < \text{rang}(x_{n-1}) < \text{rang}(x_0).$$

1.2 Algorithme de Bellman

L'algorithme de Bellman permet de déterminer le plus court chemin entre deux sommets d'un graphe sans circuit ; les valuations peuvent être négatives.

Algorithme de Bellman

Entrées : un graphe G acyclique dont les sommets sont triés dans l'ordre topologique
Sorties : les plus courts chemin à partir d'un sommet de rang 0
Variables : $d(i)$ distance de 1 à i ;
$P(i)$ prédecesseur de i
Initialisation : $d(1) = 0$;
$P(1) = \emptyset$;
pour $i = 2, \cdots ,n$ **faire**
  $d(i) = +\infty$;
  $P(i) = \emptyset$;
fin

```
1 début
2   pour i = 2, ··· ,n faire
3     d(i) = min_{j∈N⁻(i)} (d(j) + f(i,j)) ;
4     j0 = prédécesseur de i fournissant la valeur de d(i) ci-desssous ;
5     P(i) = j0 ;
6   fin
7 fin
```

Remarque. L'algorithme de Bellman permet aussi de déterminer les **plus longs chemins** (i.e. les chemins de longueur maximale) dans un graphe. Dans ce cas il suffit de modifier l'instruction

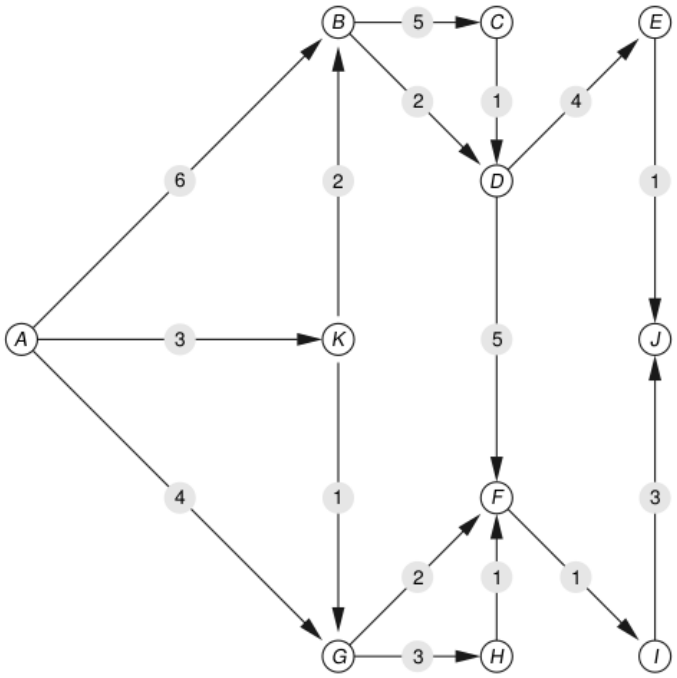
$$d(i) = \min_{j \in N^-(i)} (d(j) + f(j,i))$$

en

$$d(i) = \max_{j \in N^-(i)} (d(j) + f(j,i))$$

et d'initialiser les distances à $-\infty$ au lieu de $+\infty$.

Exercice 2. Algorithme de Bellman
Déterminer le chemin de poids minimal entre A et J du graphe suivant en utilisant l'algorithme de Bellman. N'oubliez pas de trier les sommets selon leur rang !



2 Ordonnancement : Méthode des Potentiels Metra (MPM)

2.1 Le problème

La réalisation d'une recette de cuisine, la construction d'un pont ou d'une maison, l'élaboration d'un logiciel, le fonctionnement d'une chaîne de fabrication , etc., demandent une coordination d'un ensemble complexe « d'unités de travail ».

Les problèmes d'ordonnancement sont de ce type. Ils se présentent sous la forme d'un objectif que l'on souhaite atteindre et dont la réalisation demande l'exécution d'un certains nombre de tâches soumises à de nombreuses contraintes (durée, matériels, moyen humains, météo, etc.).

Nous allons étudier ici le cas particulier de base : les seules contraintes sont du type :

- ➢ la tâche j ne peut commencer que si la tâche i est terminée ou achevée ;
- ➢ la durée de chaque tâche est une durée certaine.

Problème : La réalisation d'un projet nécessite un certain nombre de tâches dont les durées et les contraintes d'antériorité (ou de postériorité) sont les suivantes :

Tâches	**Durée**	Tâches antérieures
A	7	–
B	3	A
C	1	B
D	8	A
E	2	D, C
F	1	D, C
G	1	D, C
H	3	F
I	2	H
J	1	E, G, I

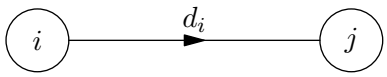
Pour cela, étudions la représentation MPM (*Méthode des Potentiels Metra*).

Remarque. Metra était, dans les années 1970, une société franco-britannique, associé à la SIA (Société d'Informatique Appliquée) et à la SEMA (Société de Mathématiques Appliquées) où beaucoup de chercheurs français de très haut niveau ont travaillé.

2.2 Description de la méthode

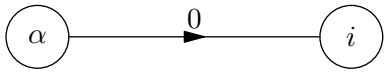
On représente le problème par un graphe tel que :

1. On associe à chaque tâche un sommet du graphe.
2. On définit l'arc $(i; j)$ de valuation ou de longueur d_i si la tâche i a une durée d_i et si la tâche i doit être terminée avant que la tâche j ne puisse commencer.

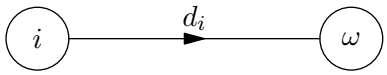
   

   Ce graphe se lisant : la tâche i a une durée d_i et la tâche j ne peut démarrer qu'après la réalisation de la tâche i.
3. On introduit deux tâches fictives, la tâche début des travaux représentée par le sommet α et la tâche fin des travaux représentée par le sommet ω.

   Ces deux tâches ont chacune une durée nulle.
4. On relie le sommet α par un arc de valuation nulle à tout sommet i représentant une tâche ne possédant pas de contrainte d'antériorité.

5. On relie le sommet i au sommet ω par un arc de valuation égale à la durée de la tâche i si la tâche i ne possède pas de contrainte de postériorité.

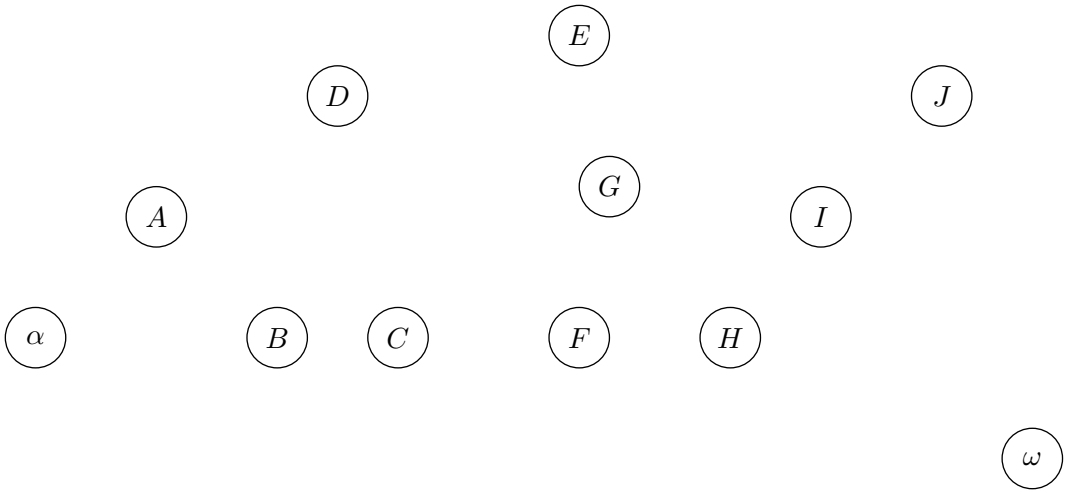
   

Exercice 3. Expliquer pourquoi ce graphe ne doit pas admettre de circuit.

2.3 Résolution

Exercice 4.

1. Tracer les arêtes du graphe ci-dessous afin qu'il représente le projet.



Remarque. Un graphe représentant un tel projet s'appelle un graphe *potentiels-tâches*.

Supposons que le projet débute à la date $t_\alpha = 0$. Notons t_i la date *au plus tôt* à laquelle peut commencer la tâche i.

2. (a) Montrer que $t_C = 10$. Quelle est la valeur de t_F ?
   (b) Interpréter alors le problème du calcul des valeurs t_i en terme de problème de chemins optimal. Quel algorithme peut-être appliquer pour résoudre ce problème ?
   (c) Appliquer cet algorithme et déterminer la durée minimale t_ω du projet.

La date *au plus tard* T_i d'une tâche i est la date au plus tard à laquelle peut commencer une tâche i sans augmenter la durée t_ω du projet.

3. (a) Que valent T_ω et T_α ?
   (b) Donnez les dates au plus tard pour tous les sommets du graphe construit à la question 1.
   (c) Pour un sommet quelconque i, exprimez T_i en fonction de t_ω et de la valeur d'un plus long chemin de i à ω.
   (d) Quel algorithme pouvez vous utiliser pour calculer les T_i ?

La marge m_i de la tâche i est la différence entre la date au plus tard et la date au plus tôt :

$$m_i = T_i - t_i$$

— Une tâche de marge nulle est une tâche critique.
— Un chemin critique est un chemin de α à ω n'empruntant que des tâches critiques.

4. (a) Donner un chemin critique.
   (b) De combien de jours au maximum peut-on retarder la tâche E sans retarder l'ensemble du projet ?

Remarque. Un graphe potentiels-tâches possède toujours au moins un chemin critique.

3 À vous de jouer

Exercice 5. Les tâches pour réaliser un projet commun de mathématiques et de synthèse d'images sont énumérées dans le tableau ci-dessous où figurent également les contraintes d'antériorité et la durée des tâches.

Tâches		durée en jours	tâches antérieures
A	comprendre le sujet	2	–
B	choisir les structures de données	3	A
C	écrire et implémenter l'algorithme 1	1	A, B
D	écrire et implémenter l'algorithme 2	1	A, B
E	écrire et implémenter l'algorithme 3	1	A, B, C, D
F	tester le programme	2	C, D, E
G	faire la partie théorique	3	A
H	rédiger le rapport	3	C, D, E, G

1. Donner les deux représentations possibles du projet.
2. Combien de jours à l'avance faut-il commencer le projet ?
3. Sur quelle(s) tâche(s) ne faut-il pas prendre de retard ?